

Implementation of Phong's Lighting and Different Types of Shading.

HARSH PRITEHSKUMAR MISTRY, Indraprastha Institute of Information Technology, India

This report details the implementation of lighting and shading techniques in OpenGL as part of Assignment 3 for CSE 333/533 - Monsoon 2024. The project focuses on generating normals for a sphere, applying Gouraud and Phong shading with a point light source, and implementing a spotlight with smooth, soft edges. Screenshots demonstrating the implementation are included.

ACM Reference Format:

Harsh Pritehskumar Mistry. 2024. Implementation of Phong's Lighting and Different Types of Shading.. 1, 1 (October 2024), 9 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

This assignment aimed to extend a basic OpenGL program that renders a sphere with no shading or lighting. We implemented normal generation, lighting models, and spotlight techniques and visualized the effects through screenshots. The assignment emphasizes the importance of proper lighting and shading in rendering 3D objects realistically.

2 NORMALS GENERATION AND VISUALIZATION

2.1 Generating Normals

The given code uses spherical coordinates to generate vertices for a sphere. Normals for the sphere are computed by normalizing the sphere's surface position for each vertex. For every vertex (x, y, z) , the normal vector $\mathbf{n}$ is calculated as:

$$\mathbf{n} = \frac{(x, y, z)}{\|\mathbf{n}\|}$$

where the normal is normalized to ensure it is a unit vector. This was implemented using the 'glm::normalize()' function on the computed coordinates. The generated normals are then stored in an array alongside the sphere's vertices.

2.2 Vertex and Normal Arrays

The vertices and normals are stored in arrays:

```
GLfloat *sphere_vertices = new GLfloat[nTheta * nPhi * 3];  
GLfloat *sphere_normals = new GLfloat[nTheta * nPhi * 3];
```

Each vertex's corresponding normal is calculated and stored in the normal array.

Author's address: Harsh Pritehskumar Mistry, harsh22200@iiitd.ac.in, Indraprastha Institute of Information Technology, Delhi, Delhi, India.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2024/10-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

2.3 Sending Normals to OpenGL Shaders

Vertex Buffer Objects (VBOs) are created to send both vertices and normals to the shaders:

```
glGenBuffers(1, &vertex_VBO);
glBindBuffer(GL_ARRAY_BUFFER, vertex_VBO);
glBufferData(GL_ARRAY_BUFFER, nTheta * nPhi * 3 * sizeof(GLfloat), sphere_vertices, GL_STATIC_DRAW);

glGenBuffers(1, &normal_VBO);
glBindBuffer(GL_ARRAY_BUFFER, normal_VBO);
glBufferData(GL_ARRAY_BUFFER, nTheta * nPhi * 3 * sizeof(GLfloat), sphere_normals, GL_STATIC_DRAW);
```

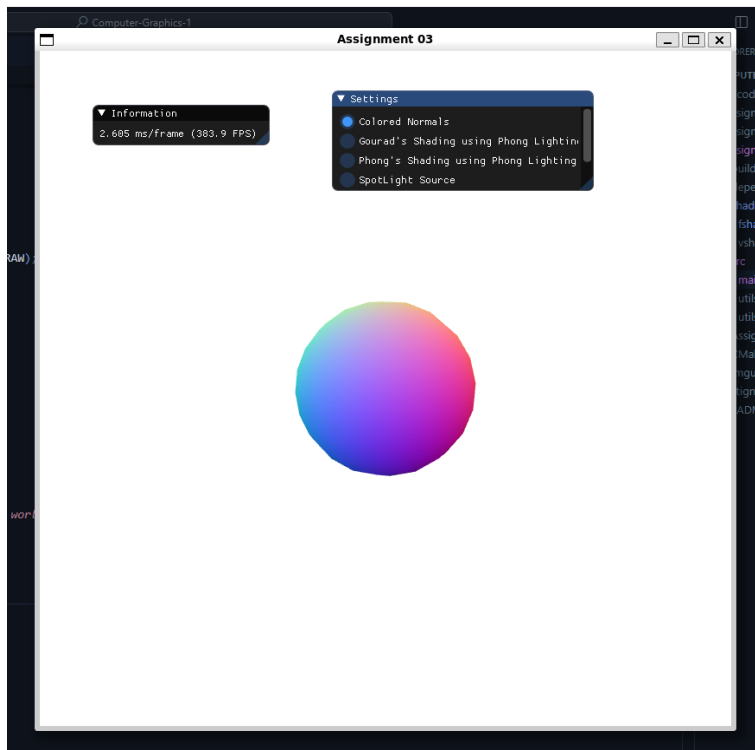
These VBOs are then bound to the shader attributes. For example, the normals are passed to the shader variable 'vNormal' as follows:

```
int vNormal_attrib = glGetAttribLocation(program, "vNormal");
glEnableVertexAttribArray(vNormal_attrib);
glVertexAttribPointer(vNormal_attrib, 3, GL_FLOAT, GL_FALSE, 0, 0);
```

After binding the VBOs, I changed the vertex shader file to use those normals to generate colours for every vertex.

2.4 Screenshot of the Output

Below is the screenshot after using the normal to generate colours for the sphere:-



3 CREATION OF POINT LIGHT SOURCE AND IMPLEMENTATION OF PHONG AND GOURAUD SHADING

3.1 Creation of Point Light Source

To create a point light source, I changed the vertex shader and passed these inputs from my C++ Program:-

```
glm::vec3 lightPos = glm::vec3(10.0f, -10.0f, 10.0f); // Light position
glm::vec3 cameraPos = glm::vec3(10.0f, -10.0f, 10.0f); // Camera position
glm::vec3 lightColor = glm::vec3(1.0f, 1.0f, 1.0f); // White light
glm::vec3 objectColor = glm::vec3(0.6f, 0.2f, 0.8f); // Purple object
```

I used these variables defined in my vertex shader program:-

```
// Uniforms for light
uniform vec3 lightPos; // Position of the point light source
uniform vec3 lightColor; // Color of the light
uniform vec3 viewPos; // Camera/viewer position
// Uniforms for material
uniform vec3 objectColor; // Color of the object (used in diffuse lighting)
```

3.2 Gouraud Shading using Phong Lighting

For Gouraud shading, we computed the lighting in the vertex shader, which includes the ambient, diffuse, and specular components. We used the Phong lighting model with a specular exponent of 32. The result was passed to the fragment shader via interpolation.

These are the changes I made to the vertex shader code:-

```
// Ambient lighting
float ambientStrength = 0.1;
vec3 ambient = ambientStrength * lightColor;

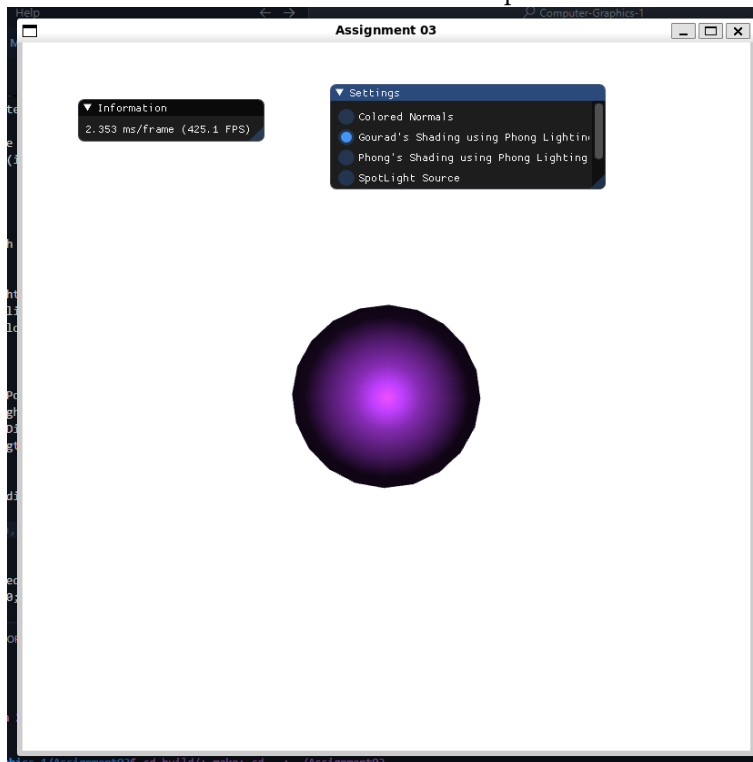
// Diffuse lighting
vec3 lightDir = normalize(lightPos - FragPos);
float diff = max(dot(normal, lightDir), 0.0);
vec3 diffuse = diff * lightColor;

// Specular lighting
float specularStrength = 0.5;
vec3 viewDir = normalize(viewPos - FragPos);
vec3 reflectDir = reflect(-lightDir, normal);
float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32); // Shininess factor
vec3 specular = specularStrength * spec * lightColor;

// Combine all components
vec3 finalColor = (ambient + diffuse + specular) * objectColor;

fColor = finalColor;
```

Attached below is the screenshot of the output:-



3.3 Phong Shading using Phong Lighting

Phong shading was implemented by moving the lighting computations to the fragment shader. This approach provides smoother and more accurate shading by calculating the lighting at each pixel.

This below is the code I added in Fragment shader to achieve Phong Shading:-

```
// Ambient lighting
float ambientStrength = 0.1;
vec3 ambient = ambientStrength * lightColor;

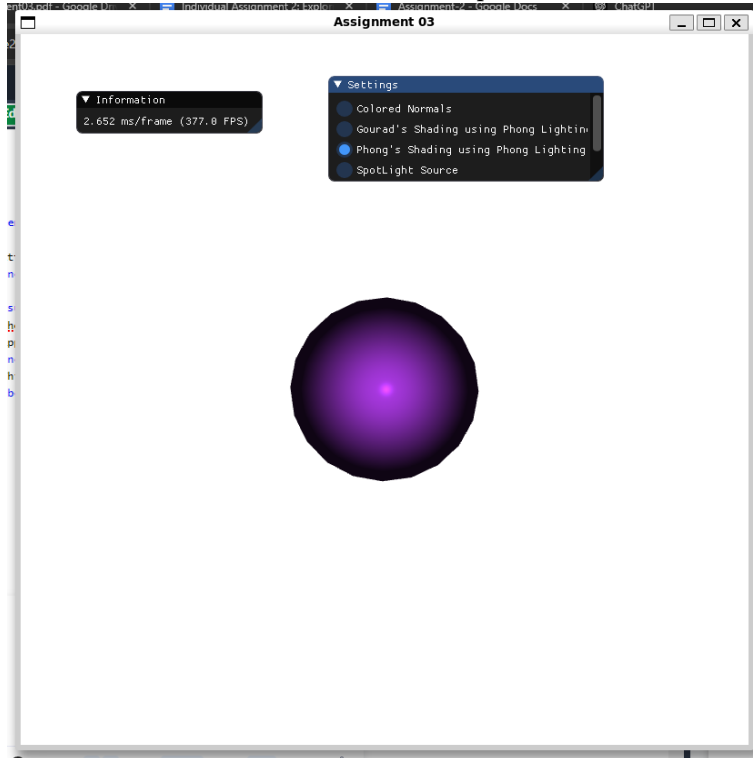
// Diffuse lighting
vec3 lightDir = normalize(lightPos - FragPos);
float diff = max(dot(normal, lightDir), 0.0);
vec3 diffuse = diff * lightColor;

// Specular lighting
float specularStrength = 0.5;
vec3 viewDir = normalize(viewPos - FragPos);
vec3 reflectDir = reflect(-lightDir, normal);
float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32); // Shininess factor
vec3 specular = specularStrength * spec * lightColor;
```

```
// Combine all components
vec3 finalColor = (ambient + diffuse + specular) * objectColor;

FragColor = vec4(finalColor, 1.0); // Set the final output color
```

Attached below is the screenshot of the output:-



4 SPOTLIGHT IMPLEMENTATION

4.1 Hard-edged Spotlight (Question 3a)

To implement the spotlight with hard edges, we created a function `spotLight()` that computes lighting only for fragments within a defined cone of illumination. The angle between the spotlight direction and the vector from the fragment to the light source characterizes this cone. If a fragment falls outside the cutoff angle, it receives only ambient lighting; otherwise, it is fully lit.

First, using the dot product, we calculate the vector from the fragment's position to the light source, `lightDir`, and the angle between this vector and the spotlight direction. The dot product determines whether the fragment is inside the cone of illumination:

$$\theta = \text{dot}(\text{lightDir}, \text{spotlightDirection})$$

The cutoff angle is defined by `innerCutoffAngle`. If the angle θ exceeds this cutoff, the fragment receives ambient, diffuse, and specular lighting components based on the Phong lighting model.

- **Ambient Lighting:** A constant ambient strength is multiplied by the light colour.

- **Diffuse Lighting:** Calculated as the dot product between the surface normal and the light direction, clamped to a minimum of 0.
- **Specular Lighting:** The reflection of the light is computed using the `reflect()` function, and the specular intensity is calculated using a shininess exponent of 32.

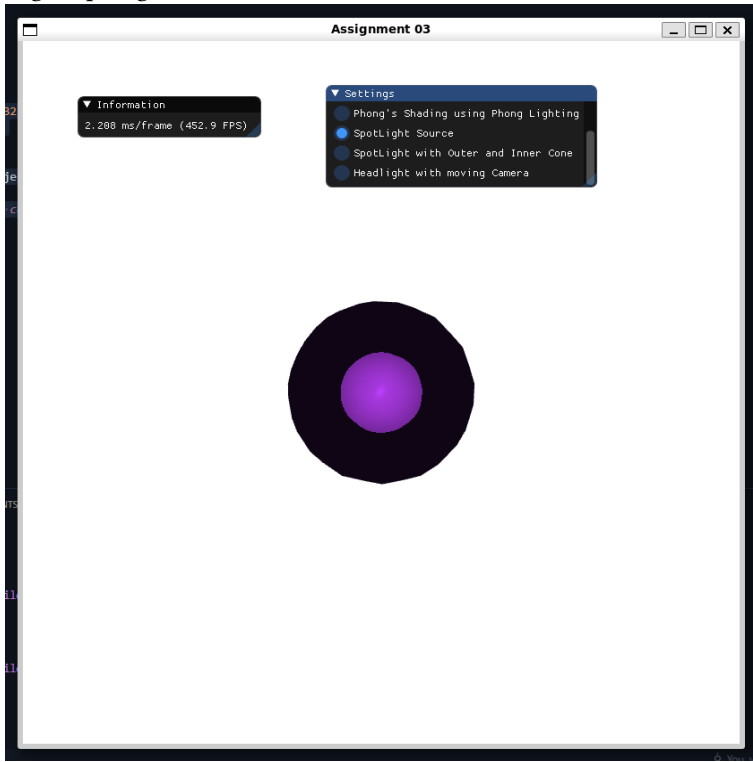
The final colour is computed by combining the ambient, diffuse, and specular components, scaled by the object's colour:

$$\text{finalColor} = (\text{ambient} + \text{diffuse} + \text{specular}) \times \text{objectColor}$$

If the fragment falls outside the cone, only ambient lighting is applied:

$$\text{finalColor} = \text{ambient} \times \text{objectColor}$$

This creates a sharp transition between fully lit and unlit areas. The figure below shows the effect of the hard-edged spotlight.



4.2 Smooth-edged Spotlight (Question 3b)

In the `spotLightwithShadow()` function, we extend the spotlight implementation to include smooth, soft edges. This is achieved by introducing an outer cutoff angle, `cutoffAngle`, and an inner cutoff angle, `innerCutoffAngle`. Fragments between the two angles are partially illuminated, with the intensity of the light smoothly interpolated based on the angle.

The intensity of the light is calculated as:

$$\text{intensity} = \frac{\theta - \text{cutoffAngle}}{\text{innerCutoffAngle} - \text{cutoffAngle}}$$

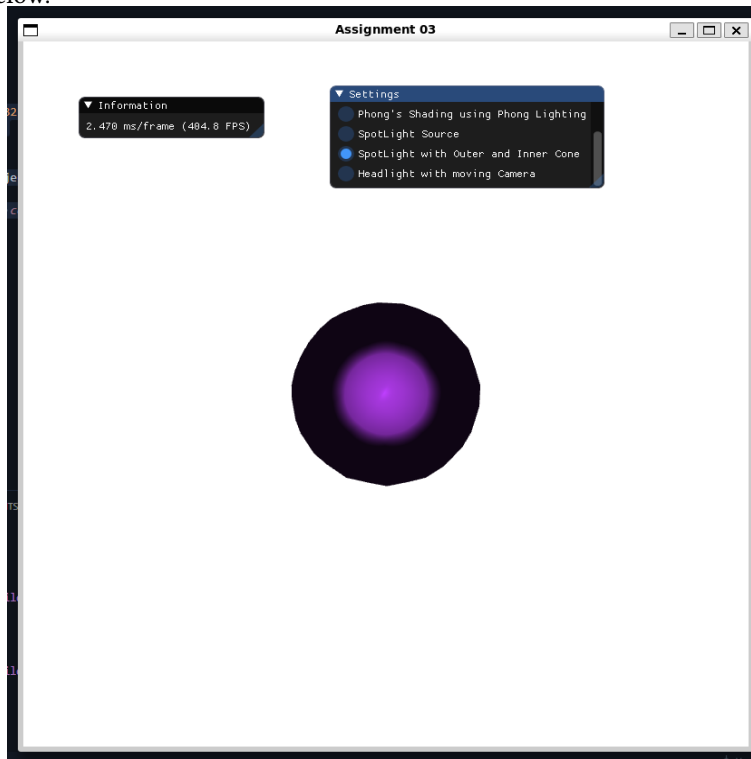
The intensity is then clamped between 0 and 1 using the `clamp()` function. This ensures a smooth transition from fully lit (intensity 1) to unlit (intensity 0).

- **Diffuse and Specular Lighting Modifications:** The diffuse and specular components are scaled by the intensity factor, which ensures that fragments closer to the inner cutoff angle receive more light, while fragments nearer the outer cutoff receive less light:

```
diffuse* = intensity
```

```
specular* = intensity
```

As with the hard-edged spotlight, ambient lighting is constant, but the diffuse and specular components are scaled by the intensity. This creates a soft transition between fully lit and unlit areas, as shown in the Figure below.



If a fragment falls outside the outer cutoff angle, it only receives ambient lighting. This technique allows for more realistic lighting where the spotlight transitions smoothly rather than having a rigid boundary.

5 BONUS: HEADLIGHT IMPLEMENTATION

As an extension of the spotlight implementation, I added a headlight, a light source that moves with the camera. The headlight's position and direction are tied to the camera's view position and direction, making the light source follow the viewer's movements. This implementation allows the light to illuminate the objects directly in front of the camera, mimicking a real-world headlight.

5.1 Headlight Implementation

The function `headlight()` calculates the light's position and direction relative to the camera. The light's position is set to the camera's position (`viewPos`), and the direction is the normalized vector from the fragment to the camera:

$$\text{lightDir} = \text{normalize}(\text{lightPos} - \text{FragPos})$$

We then calculate the angle θ between the spotlight and light direction, just like in the spotlight implementation. If the angle exceeds the cutoff defined by `innerCutoffAngle`, the fragment receives ambient, diffuse, and specular lighting based on the Phong lighting model: - **Ambient Lighting**: A constant ambient strength multiplied by the light colour. - **Diffuse Lighting**: Calculated as the dot product between the surface normal and the light direction, clamped to a minimum of 0. - **Specular Lighting**: The reflection of the light is computed using the `reflect()` function, and the specular intensity is calculated using a shininess exponent of 32.

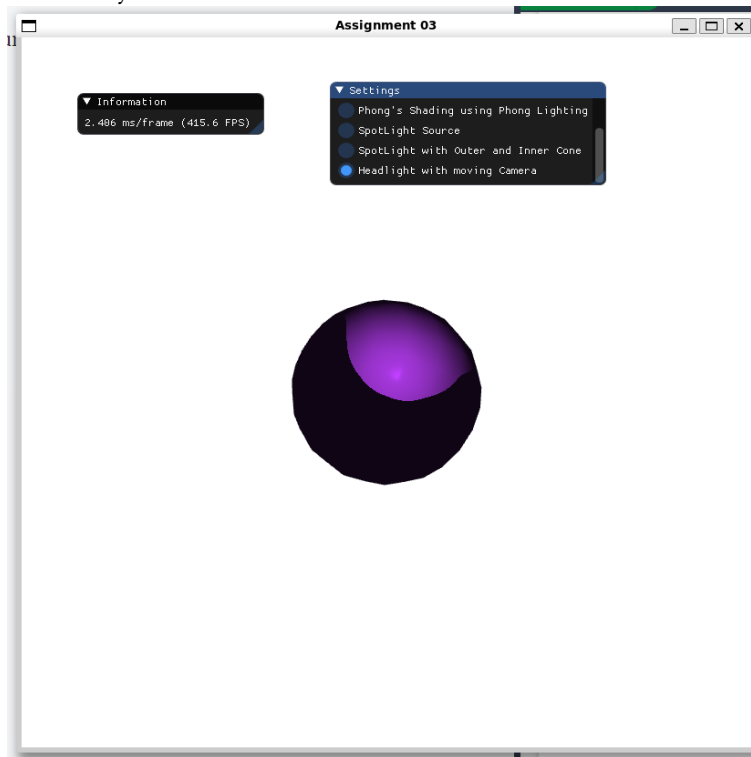
The final colour is a combination of these components:

$$\text{finalColor} = (\text{ambient} + \text{diffuse} + \text{specular}) \times \text{objectColor}$$

If the angle θ does not exceed the cutoff, only ambient lighting is applied:

$$\text{finalColor} = \text{ambient} \times \text{objectColor}$$

This ensures that fragments outside the headlight's cone of illumination still receive minimal ambient light. The image below demonstrates the effect of the headlight, where the light follows the camera and illuminates the scene directly ahead.



5.2 Headlight Behavior

Since the headlight moves with the camera, it simulates a light source fixed to the viewer's perspective. This is especially useful for navigating scenes in low-light conditions or highlighting objects directly in front of the camera. The implementation uses the same Phong lighting components as in the spotlight, with the added functionality of dynamically updating the light position and direction based on the camera's movement.

The keys used to achieve such functionality are:- W for moving the camera forward - S for moving the camera backward - A for moving the camera left - D for moving the camera right - Q for moving the camera upwards - E for moving the camera downwards

```
// Camera movement W - Forward, S - Backward, A - Left, D - Right, Q - Up, E - Down
if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
    cameraPos += cameraSpeed * glm::normalize(glm::vec3(viewT[0][2], viewT[1][2], viewT[2][2]));
if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
    cameraPos -= cameraSpeed * glm::normalize(glm::vec3(viewT[0][2], viewT[1][2], viewT[2][2]));
if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
    cameraPos -= cameraSpeed * glm::normalize(glm::vec3(viewT[0][0], viewT[1][0], viewT[2][0]));
if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
    cameraPos += cameraSpeed * glm::normalize(glm::vec3(viewT[0][0], viewT[1][0], viewT[2][0]));
if (glfwGetKey(window, GLFW_KEY_Q) == GLFW_PRESS)
    cameraPos -= cameraSpeed * glm::normalize(glm::vec3(viewT[0][1], viewT[1][1], viewT[2][1]));
if (glfwGetKey(window, GLFW_KEY_E) == GLFW_PRESS)
    cameraPos += cameraSpeed * glm::normalize(glm::vec3(viewT[0][1], viewT[1][1], viewT[2][1]));

glUniform3fv(glGetUniformLocation(shaderProgram, "viewPos"), 1,
             glm::value_ptr(cameraPos)); // Camera/view position
```